



Escuela Politécnica Superior

Grado en Ciencia e Ingeniería de Datos

Bases de Datos: Integración y Arquitectura

Práctica 1

Generación de datos sintéticos y almacenamiento de información

Francisco Jurado, francisco.jurado@uam.es

Ámbar Tenorio Fornés, ambartenorio@uam.es

Contenidos

1. Creación de datos sintéticos
2. Generación masiva de datos
3. Almacenamiento en ficheros
4. Almacenamiento en sistemas gestores de BBDD

Contenidos

- 1. Creación de datos sintéticos**
2. Generación masiva de datos
3. Almacenamiento en ficheros
4. Almacenamiento en sistemas gestores de BBDD

¿Qué son los datos sintéticos?

- La Agencia Española de Protección de Datos define los datos sintéticos como ***'datos generados artificialmente, a diferencia de los datos reales que se recopilan de la realidad'***.
 - <https://www.aepd.es/prensa-y-comunicacion/blog/datos-sinteticos-y-proteccion-de-datos>
- Cuando no existen o no se cuentan con datos reales, los datos sintéticos mantienen las características y propiedades de los datos reales.
- Resultan especialmente útiles para en las fases de desarrollo, prueba y validación.

Creación de datos sintéticos en Python con Faker

- **Faker**

- Librería para generar datos sintéticos disponible en varios lenguajes de programación
- Para Python: <https://github.com/joke2k/faker>

- Lo instalaremos con:

```
!pip install Faker
```

- Y ya se podrá importar e instanciar un objeto Faker del siguiente modo:

```
from faker import Faker
fake = Faker('es_ES') # Puede establecerse un `locale`
                       # para que los datos se generen
                       # de acuerdo a él
```

Creación de datos sintéticos en Python con Faker

- Faker dispone de un amplio conjunto de métodos para generar información aleatoria. Veamos algunos...

```
fake.first_name()
fake.last_name()
fake.name_female()
fake.phone_number()
fake.address()
fake.city()
fake.text() # Lorem Ipsum (Latín)
fake.sentences()
fake.job()
fake.company()
fake.ascii_company_email()
[fake.ascii_free_email() for _ in range(5)]
fake.ipv4_private()
```

Proveedores para la generación de datos

- ¿Y si necesito algo que Faker no genere? → proveedores (*providers*)
- Pueden encontrarse proveedores concretos en las siguientes listas:
 - Proveedores estándar, incluidos por defecto.
 - <https://faker.readthedocs.io/en/stable/providers.html>
 - Community Providers, desarrollados por la comunidad.
 - <https://faker.readthedocs.io/en/stable/communityproviders.html>
 - En Github pueden encontrarse proveedores para datos concretos.
 - Se recomienda ser cauto y revisar el código antes de emplearlos.

Proveedores para la generación de datos

- Si los **providers** estándar o los proporcionados por la comunidad, no proporcionan algo que resulte necesario, pueden implementarse nuevos
 - extendiendo la clase **BaseProvider** y
 - añadiéndolo al objeto **Faker**.
- A modo de ejemplo, veamos el código que implementa un proveedor para generar números de DNI.

Generación de datos sintéticos y almacenamiento de información

Creación de datos sintéticos

```
from faker.providers import BaseProvider

class DNIProvider(BaseProvider):
    """ Provider para generar DNI """
    __letters = 'TRWAGMYFPDXBNJZSQVHLCKE' # letras válidas del dígito de control

    def dni_number(self) -> int:
        """ Genera un número de DNI con un máximo de 8 dígitos """
        return fake.unique.random_int(min=111111, max=99999999) # número de 8 dígitos
    def dni_control_letter(self, num):
        """ Genera la letra de control para el número de DNI proporcionado por parámetro """
        return self.__letters[num % 23] # la letra de control

    def dni(self) -> str:
        """ Genera una cadena con los ocho dígitos del número de dni y la letra de
            control separados por un guión"""
        num = self.dni_number()
        control = self.dni_control_letter(num)
        return f'{num:08d}-{control}'

fake.add_provider(DNIProvider) # Añade el provider a fake para que pueda usar sus métodos
```

- Ahora puede usarse del siguiente modo:

```
dni_number = fake.dni_number()  
dni_letter = fake.dni_control_letter(dni_number)  
  
print(f'{dni_number:08d} {dni_letter}')
```

- O bien directamente:

```
fake.dni()
```

Contenidos

1. Creación de datos sintéticos
- 2. Generación masiva de datos**
3. Almacenamiento en ficheros
4. Almacenamiento en sistemas gestores de BBDD

Generación masiva de datos

- A modo de ejemplo, generemos datos de usuarios, almacenando la información en una lista de diccionarios python.
- Fíjese en el atributo ``unique`` del generador para garantizar que cualquier valor generado sea único para esta instancia específica.

```
from tqdm import tqdm # Para mostrar una barra de progreso en los iteradores.  
    # Útil cuando la cantidad de información a generar es grande y  
    # se quiere tener información del progreso
```

```
def generate_data(how_much = 1000000) -> list:  
    ''' Genera una lista de diccionarios con datos aleatorios sobre personas.  
        Por defecto 1.000.000 entradas.'''  
    data = []
```

```
    for _ in tqdm(range(how_much), desc='Generating data'):  
        data.append({'Name': fake.name(),  
                    'Age': fake.random_int(min=18, max=80, step=1),  
                    'IdNumber': fake.unique.dni(),  
                    'Address': fake.unique.street_address(),  
                    'City': fake.city(),  
                    'State': fake.state(),  
                    'PostCode': fake.postcode(),  
                    'Phone': fake.unique.phone_number()})  
    return data
```

Generación masiva de datos

- Supongamos que se desean generar 10.000 entradas de datos ficticios.

```
data = generate_data(10000)
```

- Prueba a generar 10.000, 100.000, 1.000.000, etc.

Trabajando con DataFrame

- Ahora, podría cargarse el diccionario en un `DataFrame` de Pandas para su posterior manipulación...

```
import pandas as pd
```

```
df = pd.DataFrame(data)
```

Limitaciones de Pandas en la generación de datos

- **Consumo de memoria**
 - Pandas carga todos los datos en memoria.
 - Cuello de botella en datasets grandes (ej. `generate_data(how_much=1000000)`).
- **Rendimiento limitado**
 - No está optimizado para procesamiento paralelo ni para operaciones distribuidas.
- **Dependencia innecesaria**
 - Si solo se necesita escribir o leer datos (CSV, JSON, Parquet, etc.), usar Pandas puede ser excesivo.
- **Tiempo de carga**
 - Pandas puede tardar más en inicializar y procesar datos, especialmente si hay conversión de tipos o estructuras complejas.
- **Entornos restringidos**
 - En sistemas embebidos, microservicios o contenedores ligeros, instalar Pandas puede ser costoso en términos de tamaño y dependencias.
- **Tipos de datos limitados**
 - Pandas no soporta bien tipos complejos como listas, structs, o mapas que sí puedes usar con JSON o Parquet.

Contenidos

1. Creación de datos sintéticos
2. Generación masiva de datos
- 3. Almacenamiento en ficheros**
4. Almacenamiento en sistemas gestores de BBDD

Trabajando con CSV

- CSV (Comma Separated Values)
 - Formato de texto plano.
 - Almacena información orientada a filas.
 - Almacena datos estructurados en forma de tabla, usando comas (a veces punto y coma) para separar los valores de cada columna en cada fila.
 - Formato sencillo, universal y muy usado para importar y exportar datos entre diferentes programas y bases de datos.

Trabajando con CSV

```
import csv

def write_to_csv(data, filename):
    with open(filename, 'w') as output:
        csv_writer = csv.writer(output)
        csv_writer.writerow(data[0].keys()) # Escribimos la cabecera
        for row in data:
            csv_writer.writerow(row.values()) # Escribimos los datos

# Ejemplo de uso
data = generate_data(how_much=1000000)
write_to_csv(data, 'datosFaker.csv')
```

Trabajando con CSV

- JSON (JavaScript Object Notation)
 - Formato de texto plano.
 - Legible por humanos para almacenar e intercambiar datos estructurados.
 - Sintaxis basadas en pares clave-valor.
 - Compatible con la mayoría de los lenguajes de programación
 - Ideal para la comunicación de datos en diversas plataformas y lenguajes.
 - Utilizado en aplicaciones web para el intercambio de datos cliente servidor, como formato estándar para el intercambio de datos entre aplicaciones mediante APIs, para almacenar y organizar información en bases de datos NoSQL, archivos de configuración, etc.

Trabajando con JSON

- Generar JSON

```
import json
import pprint
```

```
pprint.pp(json.dumps(data[1:5], indent = 4)) # imprimirá los 5 primeros en
# JSON con indentación de 4
```

- Guarda el JSON a fichero

```
def write_to_json(data, filename):
    with open(filename, 'w') as output:
        json.dump(data, output)
```

```
# Ejemplo de uso
```

```
data = generate_data(how_much=1000000)
write_to_json(data, 'datosFaker.json')
```

Trabajando con Apache Parquet

- <https://parquet.apache.org/>
- Formato de archivo binario de código abierto.
- Independiente del lenguaje
- Creado para manejar formatos de datos de almacenamiento en columnas.
- Permite compresión de datos.
- Compatibles con los sistemas OLAP (Procesamiento Analítico en Línea)
 - Originalmente para su uso en Apache Hadoop (Apache Drill, Apache Hive, Apache Impala, Apache Spark), que lo adoptaron como estándar compartido para la E/S de alto rendimiento.
 - Pueden realizarse consultas analíticas con motores como Spark o DuckDB
- Integración en ecosistemas *big data*, *cloud*, *datalakes*, etc.
 - Ej. Google Cloud Storage, AWS S3 y Azure Data Lake Storage soportan Parquet para análisis y procesamiento de datos.

Trabajando con Apache Parquet

```
import pyarrow as pa
import pyarrow.parquet as pq

def write_dict_list_to_parquet(data, filename):
    # Convertir lista de diccionarios a una tabla Arrow
    table = pa.Table.from_pylist(data)

    # Escribir la tabla en formato Parquet
    pq.write_table(table, filename, compression='snappy')

# Ejemplo de uso
data = generate_data(how_much=1000000)
write_dict_list_to_parquet(data, 'datosFaker.parquet')
```

Trabajando con Apache Parquet

Posibilidad de especificar el esquema con los tipos de datos

Definir el esquema con tipos de datos específicos

```
schema = pa.schema([
    ('Name', pa.string()),
    ('Age', pa.int32()),
    ('IdNumber', pa.string()),
    ('Address', pa.string()),
    ('City', pa.string()),
    ('State', pa.string()),
    ('PostCode', pa.string()),
    ('Phone', pa.string())
])
```

Convertir lista de diccionarios a una tabla Arrow con esquema

```
table = pa.Table.from_pylist(data, schema=schema)
```

- **Control total** sobre los tipos de datos.
- Evita errores de inferencia automática.
- Facilita la interoperabilidad con otros sistemas (Spark, Hive, etc.).

Trabajando con Apache Avro

- <https://avro.apache.org/>
- Formato de serialización de datos binario, compacto y rápido.
- Creado para manejar formatos de datos de almacenamiento en filas
 - Almacena los registros completos secuencialmente → ideal para operaciones de escritura de datos de alta velocidad
- Tipos primitivos (int, string, boolean, etc.) y complejos:
 - Record: Define un objeto que puede contener otros campos y anidar objetos dentro de objetos.
 - Array: Representa colecciones de elementos del mismo tipo.
 - Map: Almacena pares clave-valor.
 - Union: Permite que un campo acepte diferentes tipos de datos (como en C).
- Diseñado para:
 - Alta eficiencia en almacenamiento y transmisión.
 - Soporte nativo para esquemas de datos evolutivos (ideal para sistemas distribuidos).
 - Usado en entornos como Apache Kafka, Hadoop y sistemas de mensajería.

Trabajando con Apache Avro

```
import fastavro
```

```
# Esquema Avro definido para los datos
```

```
schema = {  
    'name': 'User', 'type': 'record',  
    'fields': [  
        {'name': 'Name', 'type': 'string'}, {'name': 'Age', 'type': 'int'},  
        {'name': 'IdNumber', 'type': 'string'}, {'name': 'Address', 'type': 'string'},  
        {'name': 'City', 'type': 'string'}, {'name': 'State', 'type': 'string'},  
        {'name': 'PostCode', 'type': 'string'}, {'name': 'Phone', 'type': 'string'}  
    ]  
}
```

```
def write_dict_list_to_avro(data, schema, filename):
```

```
    with open(filename, 'wb') as out:
```

```
        fastavro.writer(out, schema, data)
```

```
# Ejemplo de uso
```

```
data = generate_data(how_much=1000000)
```

```
write_dict_list_to_avro(data, schema, 'datosFaker.avro')
```

Contenidos

1. Creación de datos sintéticos
2. Generación masiva de datos
3. Almacenamiento en ficheros
4. **Almacenamiento en sistemas gestores de BBDD**

Trabajando con Sistemas Gestores de Bases de Datos

- **BBDD relacionales como sqlite3.**

```
import sqlite3
```

```
def write_to_sqlite3(filename):
```

```
    con = sqlite3.connect(filename)
```

```
    cur = con.cursor()
```

```
    cur.execute("""DROP TABLE IF EXISTS users""")
```

```
    cur.execute("""CREATE TABLE users (Name text, Age integer, IdNumber integer, Address text,  
                                     City text, State text, PostCode text, Phone text)""")
```

```
    for row in data:
```

```
        cur.execute("""INSERT INTO users (Name, Age, IdNumber, Address, City, State, PostCode, Phone)
```

```
                        VALUES (?, ?, ?, ?, ?, ?, ?, ?)""",
```

```
                        (row['Name'], row['Age'], row['IdNumber'], row['Address'],
```

```
                        row['City'], row['State'], row['PostCode'], row['Phone']))
```

```
    con.commit()
```

```
    con.close()
```

```
write_to_sqlite3('datosFaker.db')
```

Trabajando con Sistemas Gestores de Bases de Datos

- BBDD NoSQL como MongoDB

```
import pymongo

def write_to_mongo(dbname):
    client = pymongo.MongoClient('mongodb://localhost:27017/')
    db = client[dbname]
    db.drop_collection('users') # just in case
    collection = db['users']

    for row in data:
        collection.insert_one(row)

    # alternatively
    # collection.insert_many(data)

    client.close()

write_to_mongo('fakeMongoDatabase')
```

OJO: ¿Qué pasa con data tras hacer 'insert'?

Si se hace procesamiento en lotes, quizá sea mejor hacer una copia en profundidad de 'data'.